Криптографія. Комп’ютерний практикум №4

Вивчення криптосистеми RSA та алгоритму електронного підпису; ознайомлення з методами генерації параметрів для асиметричних криптосистем

Коротка Ксенія та Тимчак Анастасія, ФБ-43

Мета роботи

Ознайомлення з тестами перевірки чисел на простоту і методами генерації ключів для асиметричної криптосистеми типу RSA; практичне ознайомлення з системою захисту інформації на основі криптосхеми RSA, організація з використанням цієї системи засекреченого зв’язку й електронного підпису, вивчення протоколу розсилання ключів.

Постановка задачі

1. Написати функцію пошуку випадкового простого числа заданої довжини: вбудований генератор псевдовипадкових чисел, пробні ділення та власна реалізація тесту Міллера-Рабіна.
2. Згенерувати дві пари простих p, q і p_1, q_1 довжиною щонайменше 256 біт так, щоб $pq \le p_1 q_1$ (p, q — для абонента A, p_1, q_1 — для абонента B).
3. Написати функцію генерації ключових пар RSA і побудувати ключі (e, n), d для A та (e_1, n_1), d_1 для B.
4. Реалізувати шифрування, розшифрування, створення та перевірку цифрового підпису; перевірити їх на випадковому повідомленні M для A і B.
5. Реалізувати протокол конфіденційного розсилання ключів з підтвердженням справжності і перевірити його для випадкового $0 < k < n$.
6. Перевірити кожну операцію у взаємодії з тестовим середовищем http://asymcryptwebservice.appspot.com/?section=rsa.

Окремих варіантів у цьому практикумі не передбачено: усі числа генеруються випадково. Довжина кожного простого — 256 біт, модулів n і n_1 — 512 біт, відкрита експонента $e = 2^{16} + 1 = 65537$.

Програмна реалізація

Програма `lab4.py` (Python, лише стандартна бібліотека; вбудовані `pow` і перевірки простоти не використовуються) містить:

- `egcd`, `inverse` — розширений алгоритм Евкліда та обернений елемент за модулем;

- `mod_pow` — піднесення до степеня за модулем за схемою Горнера (біти показника від старшого до молодшого: квадрат, а для одиничного біта — ще множення);
- `trial_division` — пробні ділення на всі 168 простих, менших за 1000;
- `decompose`, `is_strong_pseudoprime`, `miller_rabin` — тест Міллера-Рабіна з k = 20 випадковими основами (ймовірність помилки $\leq 4^{-20}$);
- miller_rabin_trace — проміжні значення для основи з тесту;
- `random_prime`, `random_prime_bits` — випадкове x з інтервалу $[n_0, n_1]$, далі перебір непарних m_0, $m_0 + 2$, … до першого простого. Два старші біти кандидата одиничні, тому добуток двох таких чисел має рівно 512 біт;
- сім високорівневих процедур, кожна з яких отримує лише потрібні їй ключові дані:

Процедура	Вхід	Результат
`GenerateKeyPair(p, q)`	прості p, q	відкритий (n, e), секретний (d, p, q)
`Encrypt(M, (n, e))`	повідомлення, відкритий ключ адресата	$C = M^e \bmod n$
`Decrypt(C, (d, p, q))`	криптограма, власний секретний ключ	$M = C^d \bmod n$
`Sign(M, (d, p, q))`	повідомлення, власний секретний ключ	(M, S), $S = M^d \bmod n$
`Verify(M, S, (n, e))`	повідомлення, підпис, відкритий ключ автора	$M = S^e \bmod n$?
`SendKey(k, (d, p, q), (n₁, e₁))`	ключ, свій секретний, відкритий адресата	(k_1, S_1)
`ReceiveKey(k₁, S₁, (d₁, p₁, q₁), (n, e))`	повідомлення, свій секретний, відкритий відправника	k, S і результат перевірки

Результати друкуються в консоль. Прапорець --sample відтворює числа зі звіту, а --save зберігає results.txt, keys.json і candidates.txt.
Для взаємодії з тестовим середовищем написано server_check.py: він читає та виводить числа у шістнадцятковому вигляді (hex) і викликає процедури з lab4.py для ключа абонента A.

Хід роботи

1. Генерація простих чисел

Кожен кандидат спочатку перевіряється пробними діленнями; якщо дільник не знайдено — тестом Міллера-Рабіна. Перше число, що пройшло обидві перевірки, приймається як просте.

Число	Відкинуто кандидатів	з них пробними діленнями	з них тестом Міллера-Рабіна
p	43	36	7
q	53	41	12
p_1	16	15	1
q_1	49	39	10

$p = 10249944319780218478551253943027536095157018718676967029988606484601025510453 9$

$q = 94924324348209740335537100506204773038475645058624884035298661040595931073947$

$p_1 = 90365419907173691547750433816579595749194215074082676608687964835584340285853$

$q_1 = 11434739654258882227860255364236212303332702617769678454516846801704012175854 7$

Умова $pq \le p_1 q_1$ виконується. p, q взято для абонента A, p_1, q_1 — для B.

Кандидати, що пройшли пробні ділення, але не пройшли тест Міллера-Рабіна (кандидати, відкинуті пробними діленнями, — у `candidates.txt`):

- для p: 1024994431978021847855125394302753609515701871867696702998860648460102551044 59; 1024994431978021847855125394302753609515701871867696702998860648460102551045 03; 1024994431978021847855125394302753609515701871867696702998860648460102551045 07; 1024994431978021847855125394302753609515701871867696702998860648460102551045 13; 1024994431978021847855125394302753609515701871867696702998860648460102551045 21
- для q: 9492432434820974033553710050620477303847564505862488403529866104059593107384 3; 9492432434820974033553710050620477303847564505862488403529866104059593107386 1;

949243243482097403355371005062047730384756450586248840352986
61040595931073863; 949243243482097403355371005062047730384756
564505862488403529866104059593107386 9; 9492432434820974033553710050620477303847
5645058624884035298661040595931073869; 94924324348209740335
537100506204773038475645058624884035298661040595931073893;
949243243482097403355371005062047730384756450586248840352986
61040595931073897; 949243243482097403355371005062047730384756
564505862488403529866104059593107391 1; 94924324348209740335
537100506204773038475645058624884035298661040595931073927;
949243243482097403355371005062047730384756450586248840352986
61040595931073929; 949243243482097403355371005062047730384756
564505862488403529866104059593107393 9; 94924324348209740335
537100506204773038475645058624884035298661040595931073941

- для p_1: 90365419907173691547750433816579595749194215074082676608687964835584340285823
- для q_1: 11434739654258882227860255364236212303332702617769678454516846801704012175845316846801704012175845 3; 1143473965425888222786025536423621230333270261776967845451684680170401217584597; 11434739654258882227860255364236212303332702617769678454516846801704012175846 9; 1143473965425888222786025536423621230333270261776967845451684680170401217584 99; 11434739654258882227860255364236212303332702617769678454516846801704012175851 3; 1143473965425888222786025536423621230333270261776967845451684680170401217585 29; 11434739654258882227860255364236212303332702617769678454516846801704012175853 1; 1143473965425888222786025536423621230333270261776967845451684680170401217854 1

2. Проміжні результати тесту Міллера-Рабіна

Для наочності взято просте p_1, у якого розклад $p_1 - 1 = d \cdot 2^s$ має найбільше s серед чотирьох згенерованих ($s = 2$).

- $p_1 - 1 = d \cdot 2^s$, $s = 2$, $d = 225913549767934228869376084541448989372985537685206691521719912088960850714 63$;
- випадкова основа $x = 114670482293232406847582119352254148100865409852699300736351343390268345403 82$, $\gcd(x, p_1) = 1$;
- $x_0 = x^d \bmod p_1 = 84943062628591051208925390406749636066825840502892627886629811171619350913831 \neq \pm 1$ — переходимо до кроку 2.2;
- $r = 1$: $x_1 = x_0^2 \bmod p_1 = 90365419907173691547750433816579595749194215074082676608687964835584340285852 = p_1 - 1 \equiv -1 \pmod{p_1}$.

Отже, p_1 пройшло раунд за основою x: $x_1 = p_1 - 1$. Для відтворення прикладу виконано повторний тест за 20 основами, починаючи з x.

3. Ключі RSA для абонентів A і B

Абонент A:

- n = pq = 97296903916190751769316961086284897310483090601199753082607
  66410792905131883188107759414436058791379293278251969023428793207519504745009631833650243454 33
- φ(n) = (p − 1)(q − 1) = 97296903916190751769316961086284897310483090601199753082607666410792905131881213870083954316807580882893913450629122970470753573961393162372967588381669 48
- e = 65537
- d = e^{-1} mod φ(n) = 52809072267617090058842693788245419873219616640604184154011280679590375336773832469791970016985251988730326309601118958490855171573015490162331871511415 7

Перевірка співвідношень, необхідних для коректної роботи RSA (виконується програмою):

- n = p·q, n має 512 біт;
- gcd(e, φ(n)) = 1 — тому e оборотне за модулем φ(n);
- e·d mod φ(n) = 1;
- 1 < e < φ(n), 1 < d < φ(n);
- n ≤ n_1 — умова для протоколу розсилання ключів.

Тоді для будь-якого 0 ≤ M < n виконується $(M^e)^d \equiv M \pmod n$, що підтверджують усі подальші перевірки.

Абонент B:

- n_1 = 10333050503863140110337231158184419495187872934098471165503839722878367957728561099496132308672826272084959796831166707405210140713694009518104279025935591
- e_1 = 65537
- d_1 = 22451842344784032709950435890038624534457682008874716480274452241296971133566045889546161627371432755229627870790599327246405399123574740975934748461145 7

4. Шифрування та цифровий підпис

Відкрите повідомлення M вибиралося датчиком випадкових чисел з інтервалу 0 < M < n.

Абонент A:

- M = 424178623433640076102956487988325200296004367371672906937376
  814456707046385293508498664853193680769723207668208302528788
  667740271503115287255273734564798
- C = M^e mod n = 832375199317704277687538900160390460061265520153 60
  279854219803875410031132492951676645923005611598860677156484
  908035589249454268068651120637854483186260 94
- C^d mod n = M — розшифрування правильне;
- S = M^d mod n = 882480148878682079408933211356083529140688101524 37
  189187745375004138472880651873587070645205447488972408135395
  988877125584450074854815927755546368644505 00
- Verify(M, S) = True; для зміненого повідомлення Verify(M + 1, S) = False.

Абонент B:

- M = 309202484519167376375167079534768954868162801696641036735168
  105386201364889211398458566015272971228778389680208442155718
  554313927555943050930157469184748 0
- C = M^{e_1} mod n_1 = 696830831858025424333891142779648812212336537348 10
  540960040453109054947136419665661605622007779915997354191757
  624169268780298050440380441562275849117148 50
- C^{d_1} mod n_1 = M — розшифрування правильне;
- S = M^{d_1} mod n_1 = 507051193527408345000344757127707486860730025873067
  657950759306234923097104278574977366770017702043883285645570
  562226871190567121978020273005288053913472 2
- Verify(M, S) = True; Verify(M + 1, S) = False.

5. Протокол конфіденційного розсилання ключа A → B

Умова n ≤ n_1 виконується. Абонент A вибрав випадковий ключ 0 < k < n.

1. A: k = 833270262242042275704410215085731794994797603497384198861940
   145631978437593304978432427122378975550585050265029363627313
   317711232114319683539750233842702 7
2. A: S = k^d mod n = 270385598327985066115966604108778920357283455564 5
   541975223984729278519071739731405096497619241061958353619279
   464223746537994257545317865357491312360252 85
3. A: k_1 = k^{e_1} mod n_1 = 612257703554027578492718321855907200357922311422 79
   047444754134189246028737057312694641844515951214365217349777
   835560872852533606700947443147998108827114 11
4. A: S_1 = S^{e_1} mod n_1 = 855850239554736004109924238859483167357 4
   442792323324390096611856666431067619299224555134534317178390
   994646814426433781941204582102125427332991149745258900
5. A → B: по відкритому каналу передається (k_1, S_1).

6. B: $k = k_1^{d1} \bmod n_1$ = 833270262242042275704410215085731794994797603497384198861940145631978437593304978432427122378975550585050265029363627313317711232114319683539750233842702 7
7. B: $S = S_1^{d1} \bmod n_1$ = 270385598327985066611596660410877892035728345556455419752239847292785190717397314050964976192410619583536192794642237465379942575453178653574913123602528 5
8. B: $S^e \bmod n = k$ — підпис вірний, отриманий ключ збігається з надісланим.

Конфіденційність забезпечується відкритим ключем отримувача (e_1, n_1): k_1 і S_1 зашифровані ним, і розшифрувати їх може лише власник секретного d_1, тобто B.

Підтвердження справжності відправника забезпечується секретним ключем відправника d: $S = k^d \bmod n$ може обчислити лише A, а будь-хто перевіряє рівність $k = S^e \bmod n$ відкритим ключем A (e, n).

6. Перевірка на тестовому середовищі

Для перевірки використано ключ абонента A (у шістнадцятковому вигляді n = B9C5BC68B549ECECC1E5C2B6982436EB57E50ED1AFF64468EC309138E9F86D16C99B458B8FFD2BFBC7D93013B5F04414A5CC72FA0EE25E2A6880A07207676D59, e = 10001) і ключ сервера на 512 біт. Кожна операція перевірялась в обидва боки.

1. **Шифрування.** Сервер зашифрував нашим відкритим ключем повідомлення M = 4C6162342046422D3433204B6F726F746B612054796D6368616B («Lab4 FB-43 Korotka Tymchak»). Наша `Decrypt` повернула те саме M (рис. 2).
2. **Розшифрування.** Наша `Encrypt` зашифрувала ключем сервера M = 52534120746573743A2046422D3433202D3E20736572766572 («RSA test: FB-43 -> server»). Сервер розшифрував те саме M (рис. 3).
3. **Підпис.** Сервер підписав повідомлення; наша `Verify` з відкритим ключем сервера дала True, а для зміненого повідомлення — False (рис. 4).
4. **Перевірка підпису.** Наша `Sign` підписала повідомлення «Signed by FB-43, checked by server»; сервер повернув Verification = true (рис. 5).
5. **Надсилання ключа сервером.** Оскільки n сервера < n, сервер надіслав нам (k_1, S_1); наша `ReceiveKey` отримала k = DC4E3CB8E111DF95 і підтвердила підпис (рис. 6).
6. **Отримання ключа сервером.** Для цього напрямку потрібно $n \le$ n сервера, тому взято ключ сервера на 1024 біти (рис. 7). Наша `SendKey` сформувала (k_1, S_1) для k = 1633EF988208DB42082802F3087DE3C9; сервер отримав той самий k і повернув Verification = true (рис. 8).

Усі значення наведено у `server_check.txt`.

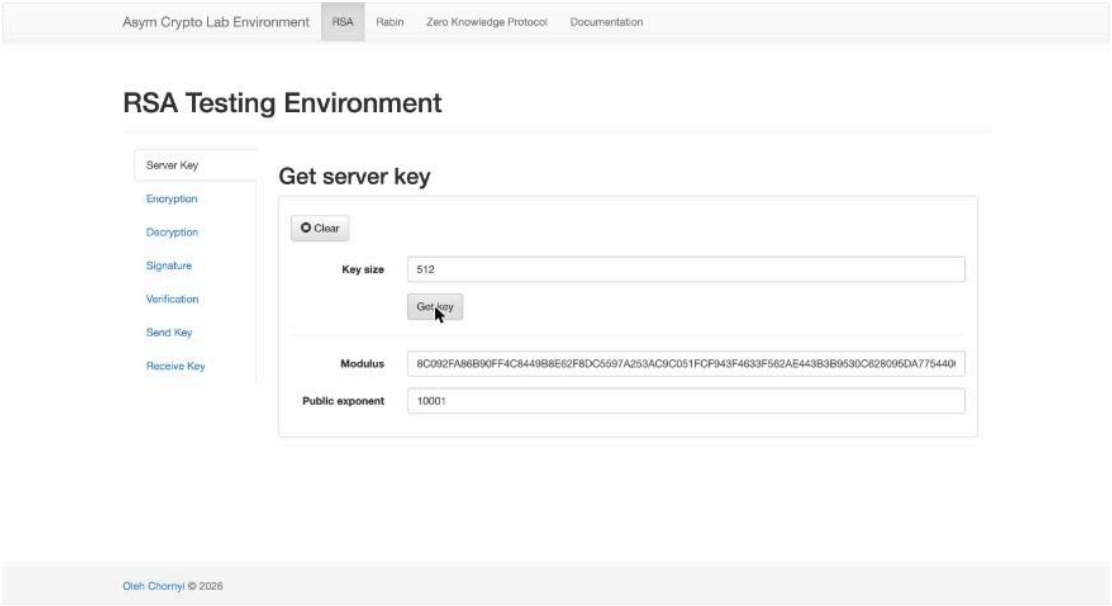


Рисунок 1: Ключ сервера (512 біт)

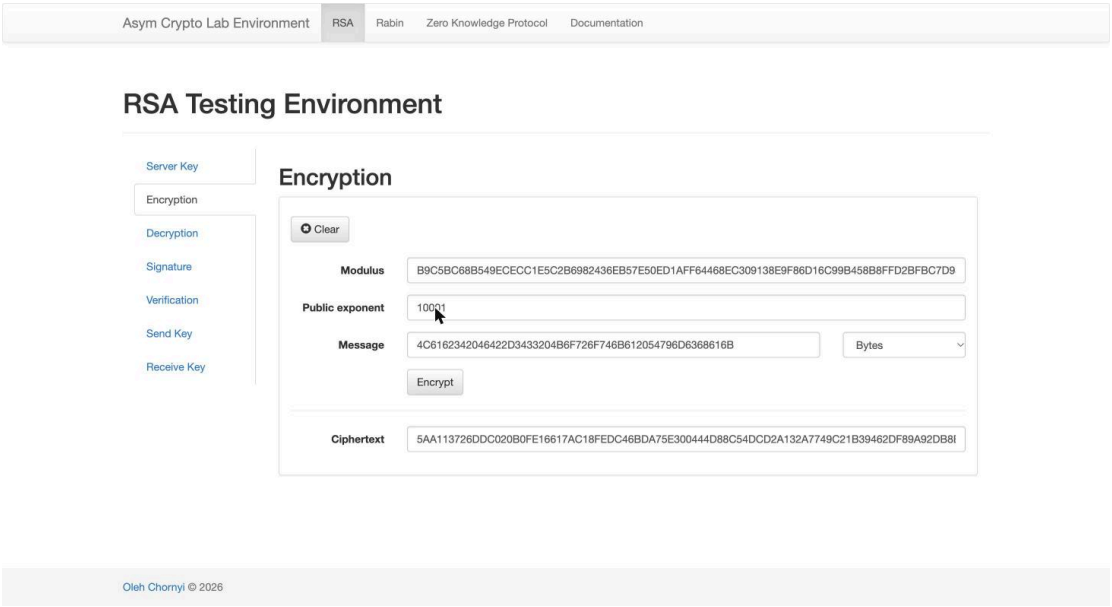


Рисунок 2: Шифрування на сервері для нашого ключа

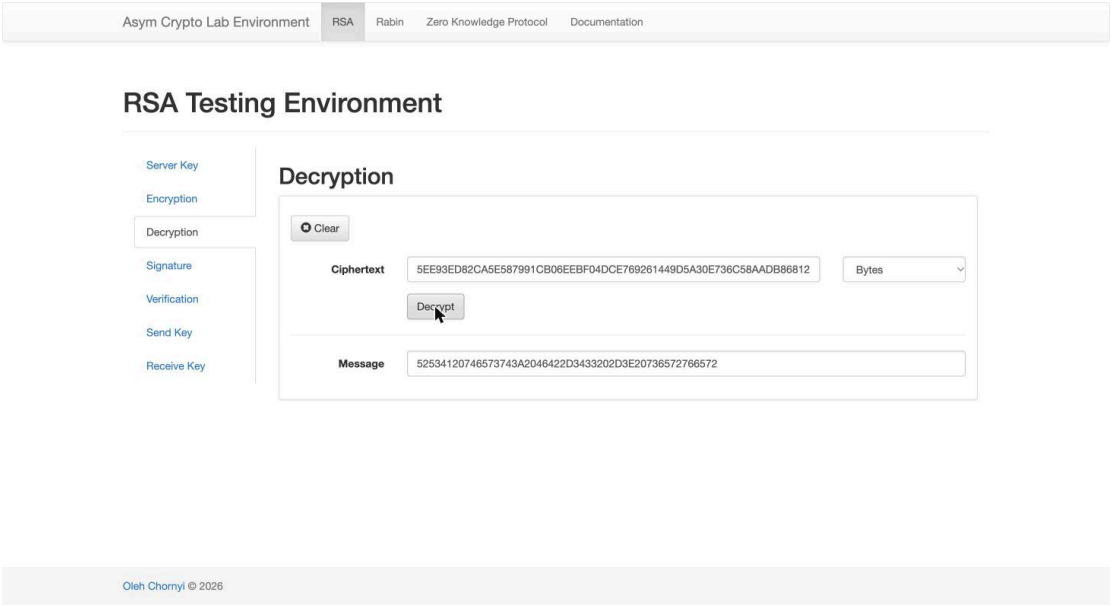


Рисунок 3: Розшифрування на сервері повідомлення, зашифрованого нашою реалізацією

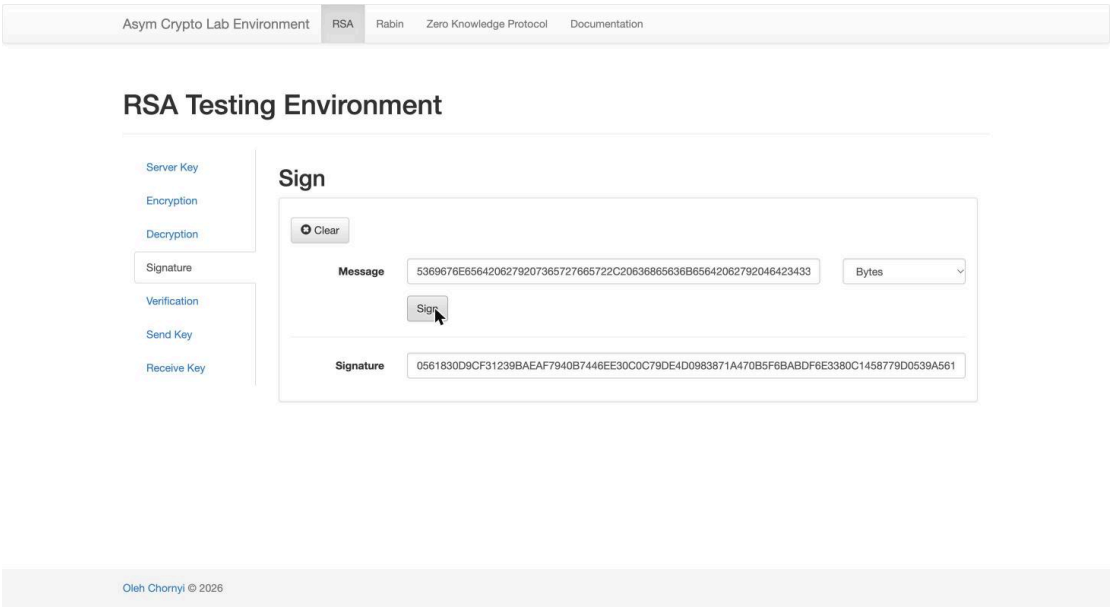


Рисунок 4: Підпис, створений сервером

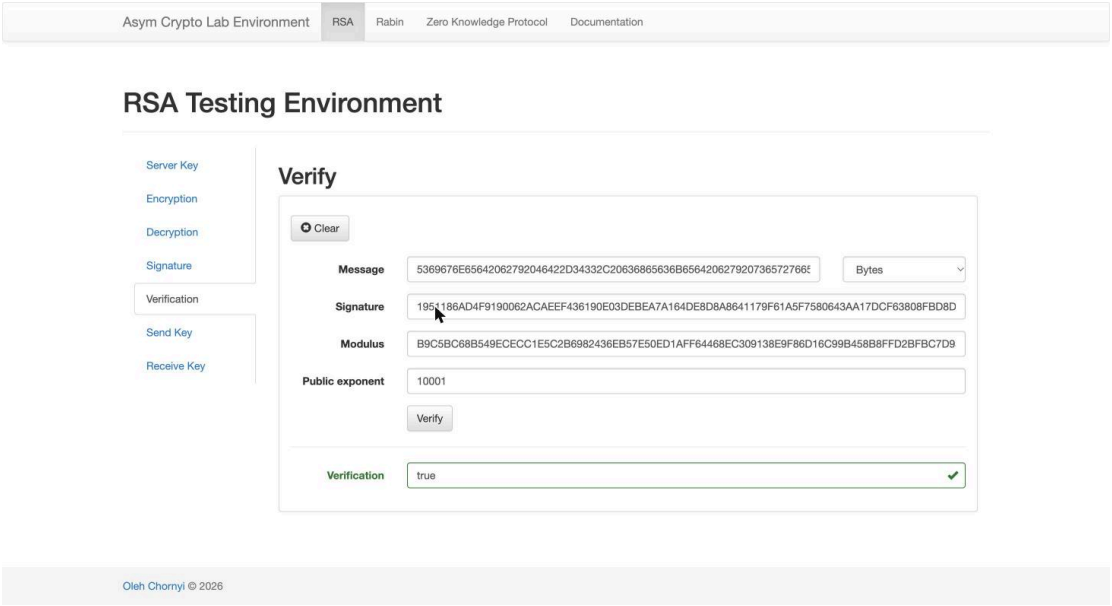


Рисунок 5: Перевірка сервером нашого підпису

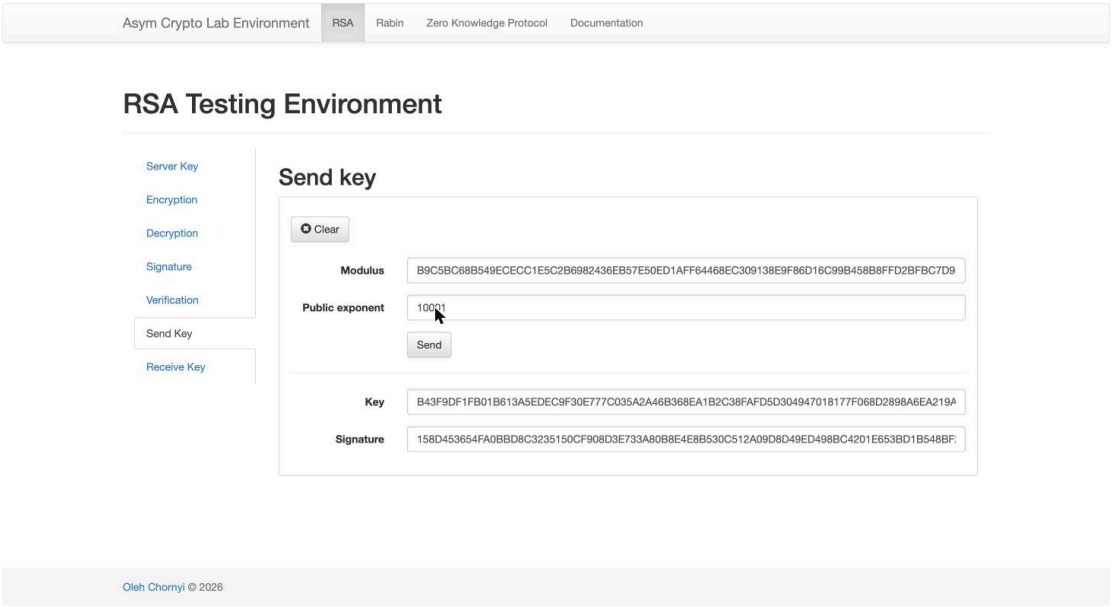


Рисунок 6: Надсилання ключа сервером

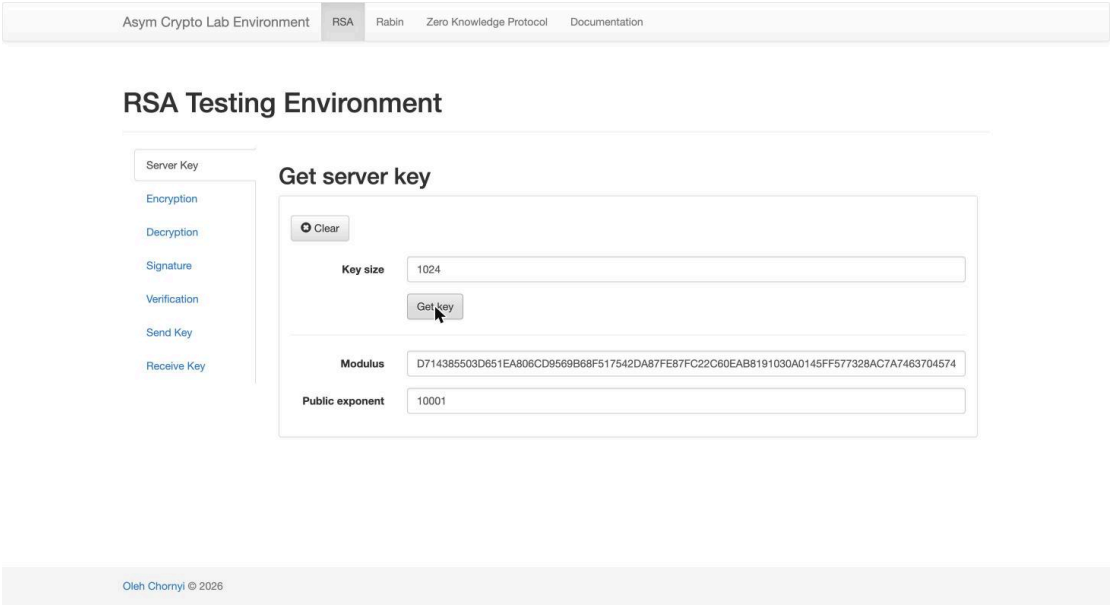


Рисунок 7: Ключ сервера на 1024 біти

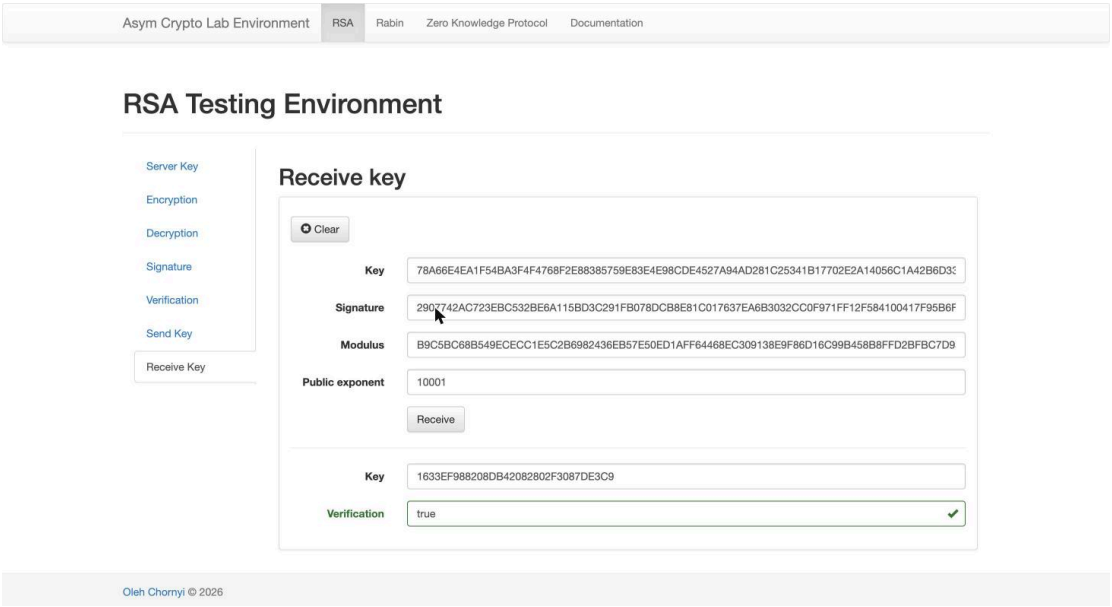


Рисунок 8: Отримання сервером ключа, надісланого нашою реалізацією

Труднощі та шляхи їх розв’язання

- **Довжина модуля.** Добуток двох 256-бітних простих може мати 511 біт. Тому в кандидатах встановлюються два старші біти: тоді $p, q \geq 3 \cdot 2^{254}$, і $n = pq$ має рівно 512 біт.
- **Умова $n \leq n_1$ у протоколі.** Для абонентів A і B вона забезпечується перестановкою пар, якщо $pq > p_1q_1$. На тестовому середовищі в одному напрямку потрібно n сервера $\leq n$, а в іншому — навпаки. Тому для отримання ключа сервером взято ключ сервера на 1024 біти, а для решти операцій — на 512 біт.

- **Формат даних тестового середовища.** Сайт працює з шістнадцятковими числами, тому для перевірки написано окремий скрипт `server_check.py`, що переводить hex у числа та назад і викликає ті самі процедури.

Відповіді на контрольні питання

1. Які перевірки виконуються програмою під час генерації простих чисел і для чого потрібна кожна з них? Спочатку — пробні ділення на прості, менші за 1000: вони дешево відкидають більшість складених чисел (у нашому запуску 131 з 161 відкинутих кандидатів). Далі — тест Міллера-Рабіна з 20 випадковими основами: для кожної основи перевіряється $\gcd(x, p) = 1$ і сильна псевдопростота. Складене число проходить одну основу з імовірністю не більше ¼, тому помилка — не більше 4^{-20}. Крім того, перевіряється довжина (два старші біти встановлені), а для пар — умова $pq \le p_1 q_1$.

2. Які умови повинні виконуватися для параметрів згенерованої пари ключів RSA? Як їх було перевірено у вашій реалізації? p і q — різні прості потрібної довжини; $n = pq$; $\gcd(e, \varphi(n)) = 1$, щоб існувало d; $e \cdot d \equiv 1 \pmod{\varphi(n)}$; $1 < e, d < \varphi(n)$. GenerateKeyPair перевіряє $p \neq q$, $1 < e < \varphi(n)$ та $\gcd(e, \varphi(n)) = 1$. Програма виводить $e \cdot d \bmod \varphi(n) = 1$.
Коректність підтверджено: Decrypt(Encrypt(M)) = M та Verify(M, S) = True для S, створеного Sign(M), для A і B та на тестовому сервері.

3. Які дані необхідні для виконання операцій Encrypt(), Decrypt(), Sign() та Verify()? Чому кожній із цих операцій не потрібні всі параметри RSA? Encrypt — повідомлення і відкритий ключ адресата (n, e); Decrypt — криптограма і власний секретний ключ (d, n = pq); Sign — повідомлення і власний секретний ключ; Verify — повідомлення, підпис і відкритий ключ автора. Операції з відкритим ключем виконує будь-хто, тому вони не можуть залежати від секретних даних. Секретний ключ передається лише до Decrypt(), Sign(), SendKey() і ReceiveKey(), де він необхідний.

4. Які властивості забезпечуються на різних етапах реалізованого протоколу передачі ключа: конфіденційність та підтвердження справжності відправника? Обчислення $S = k^d \bmod n$ секретним ключем A дає автентичність: такий підпис може створити лише A. Шифрування $k_1 = k^{e_1} \bmod n_1$ та $S_1 = S^{e_1} \bmod n_1$ відкритим ключем B дає конфіденційність: розшифрувати їх може лише B за допомогою d_1. На стороні B розшифрування за допомогою d_1 відновлює k та S, а перевірка $k = S^e \bmod n$ відкритим ключем A підтверджує відправника.

5. Які перевірки повинен виконати одержувач у реалізованому протоколі передачі ключа, перш ніж вважати отриманий ключ коректним і справжнім? Одержувач перевіряє $0 \le k_1, S_1 < n_1$ і розшифровує їх власним секретним d_1. Далі перевіряє $0 < k < n$ та $0 \le S < n$.
Після цього перевіряє підпис: $k = S^e \bmod n$ відкритим ключем A,

справжність якого має бути підтверджена. Якщо перевірка не пройдена, отриманий ключ відхиляється. Лише після успішної перевірки функція ReceiveKey() повертає (k, S, True), інакше — (None, None, False).

Висновки

Реалізовано генерацію великих випадкових простих чисел на основі пробних ділень і власного тесту Міллера-Рабіна, генерацію ключів RSA, шифрування, розшифрування, цифровий підпис і протокол конфіденційного розсилання ключа з підтвердженням справжності. Для 256-бітних простих знадобилося перевірити від 17 до 54 кандидатів; більшість складених відсіяно пробними діленнями, решту — тестом Міллера-Рабіна. Для ключів абонентів A і B виконуються всі необхідні співвідношення, а розшифрування й перевірка підпису дали правильні результати. Взаємодія з тестовим середовищем підтвердила сумісність реалізації в обидва боки для всіх операцій. Конфіденційність у протоколі забезпечує відкритий ключ отримувача, а справжність відправника — його секретний ключ.